Heirarchital Task Network language to Unity AI Class

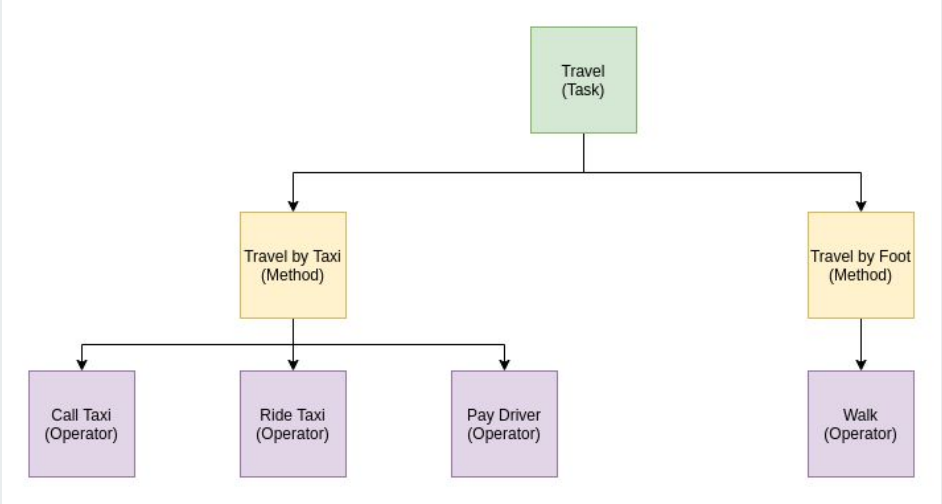
ECSE 539 - Final Project

Jiaqi Yang and Jiawei He

Language Specifications

What is a Heirarchical Task Network?

- Fancy Decision tree
- Comparison to traditional decision tree
- Forward planner
- Expand and update preconditions
- Allows more complex behavior



y

Language Specifications

DSL Specifications

- Role
- Slot
- Rule
- Action
- Sequence
- Cue

```
role OgreGuard {
  slots { player, villager, post }

  rule attackPlayer {
    when sees(player)
    choose {
      chance 60 -> chase(player);
      sequence {
        pick_rock;
        throw_at(player);
      }
    }
  }
}
```

Transformation

- Transformation environement
- AST Parser
- Top-Down Recursive Parser
-

HtnScript (Source)	Generated C# (Target)	Description
role	Class extending AiAgentBase	Defines a Unity AI component
slots	[SerializeField] GameObject fields	Exposed Unity object references
slots (binding)	slots.Set(...)	Registers slots in runtime system
rule	builder.AddRule(...)	Adds behavior rule to builder
when	Cues.* expressions	Converted into runtime condition checks
interrupt_when	Extra argument in AddRule	Interrupt condition passed separately
do	Direct statement emission	No wrapper just forwards execution
sequence	Statements.Sequence(...)	Ordered execution of actions
choose	Statements.Choose(...)	Multiple possible execution paths
chance N ->	Statements.Chance(Nf ...)	Probability-based branching
ActionStatement	Statements.Action(...)	Wraps single action
Action (e.g. chase)	Actions.*(...)	Calls runtime action methods
Condition (or)	.Or(...)	Logical OR
Condition (and)	.And(...)	Logical AND
not	Cues.Not(...)	Negation
sees(slot)	Cues.Sees("slot")	Perception check
at(slot)	Cues.At("slot")	Location check
exists(kind)	Cues.Exists("kind")	World query
TargetExpr	Targets.*(...)	Target resolution

y

Output Environment

- Unity AI Class
- Integrated with Unity APIs

```
Unity Script | 0 references
public class OgreGuardAgent : AiAgentBase
{
    [Header("Slots")]
    [SerializeField] private GameObject player;
    [SerializeField] private GameObject villager;
    [SerializeField] private GameObject post;

    2 references
    protected override void BindSlots(AiSlotRegistry slots)
    {
        slots.Set("player", player);
        slots.Set("villager", villager);
        slots.Set("post", post);
    }

    2 references
    protected override void DefineBehavior(AiBehaviorBuilder builder)
    {
        builder.SetRole("OgreGuard");
        builder.DeclareSlot("player");
        builder.DeclareSlot("villager");
        builder.DeclareSlot("post");
        builder.AddRule(
    "attackPlayer",
    Cues.Sees("player"),
    Statements.Choose(
    Statements.Chance(60f, Statements.Action(Actions.Chase("player"))),
    Statements.Option(Statements.Sequence(
    Actions.PickRock(),
    Actions.ThrowAt("player")
})
```

y

Demo!